Full-Stack Quest

Coordination développement Front et Back

Deux équipes, un contrat

Rôles front et back, contrats d'API, architecture

Rejoindre la classe

localhost:5173/rejoindre

Date

Lundi 26 octobre 2026

Horaires · salle

9 h – 13 h · 14 h – 17 h · S102

Chapitre

01 / 13

Durée

7 h

Jeu du jour

Contract Tetris N1-4

Mastère 2 Expert en développement Full Stack · A2627_0195

Le déroulé du jour

Lundi 26 octobre 2026 · 9 h – 13 h · 14 h – 17 h

Matin · 9 h 00 – 13 h 00

9 h 00	Accueil et connexion 10 min	Qui je suis, le cours en une phrase. Chacun rejoint Full-Stack Quest : QR code, code de classe, pseudo, PIN.
9 h 10	Contract Tetris N1-4 30 min	
9 h 40	Théorie 1 · Deux équipes, un contrat 55 min	
10 h 35	Atelier · Rétro-contrat 45 min	
11 h 20	Pause 10 min	
11 h 30	Atelier · Implémentation à l'aveugle 50 min	
12 h 20	Atelier · Restitution 40 min	

Après-midi · 14 h 00 – 17 h 00

14 h 00	Théorie 2 · Architectures et sujet du projet 35 min
14 h 35	Constitution des équipes 20 min
14 h 55	Cadrage du mini-jeu 50 min
15 h 45	Pause 10 min
15 h 55	Architecture cible 50 min
16 h 45	Pitch éclair 15 min

On ne vous apprend pas à coder. On vous apprend à livrer à deux.

Front et back, deux équipes, un contrat entre elles. 63 heures pour vivre les frictions du métier plutôt que d’en entendre parler.

Rejoindre la classe

01 Scanne le QR code ou ouvre l’adresse dans ton navigateur.

02 Saisis le **code de classe** écrit au tableau.

03 Choisis un **pseudo** (visible au classement) et un **code PIN** à 4 chiffres.

04 Tu arrives sur le hub : le chapitre 1 est ouvert, Contract Tetris t’attend.

Pour revenir plus tard : code de classe, pseudo, PIN. PIN oublié : l'intervenant le réinitialise depuis la console.



localhost:5173/rejoindre

Une séance, quatre temps

21 %

Jeu

Tetris à 9 h, puis le jeu du jour

19 %

Théorie

deux blocs courts

60 %

Projet

en équipe, sur votre mini-jeu

- **On joue la situation** : le jeu vous met face au problème avant qu'on le nomme.
- **On l'explique** : la théorie arrive après, quand elle répond à une question que vous vous posez.
- **On l'applique** : le projet reprend la notion le jour même.

Contract Tetris

Niveaux 1 à 4 · Solo · manches de 3 min

01 Ouvre **Contract Tetris** depuis le hub.

02 Un contrat s’affiche à gauche ; des blocs tombent. Chaque colonne est un **verdict**.

03 ← → pour déplacer, ↓ pour accélérer, `Espace` pour lâcher.

04 Enchaîne les niveaux 1 à 4. Lis l’écran de fin : chaque erreur cite la règle du contrat.

Temps de jeu

30:00

▶ Cliquer pour lancer

Compétence

Lire un contrat REST d'un coup d'œil

Où jouer

localhost:5173 → hub → Contract Tetris

Deux équipes, un contrat

Théorie 1 · 9 h 40 · 55 min

01 Le cours et ses règles

02 Front et back, qui porte quoi

03 Les frictions classiques

04 Le contrat d'API

Vous venez de juger des requêtes sans voir une ligne de code.

C’est exactement la position d’une équipe front devant une API, et d’une équipe back devant ce que le front lui enverra. Le seul terrain commun : le contrat.

Ce que vous saurez faire le 16 mars

#	Objectif du syllabus	Où vous l'exercez
1	Superviser la coordination front/back	API Diplomacy, équipes en sous-groupes
2	Gérer les interdépendances techniques	Mocks, intégration au hub
3	Définir et documenter les interfaces	Contract Tetris, contrat de votre jeu
4	Mettre en place des pipelines CI/CD multi-couches	Pipeline Tycoon, CI du projet
5	Garantir la qualité par les tests	Tests de contrat, E2E
6	Optimiser la performance	Stack Duel, test de charge
7	Résoudre les conflits d'intégration	Changement de spec, migration v2
8	Documenter et assurer la traçabilité	ADR, post-mortem, manifeste

La saison : 13 séances

01	lun. 26 oct.	7 h	**Deux équipes, un contrat**	Contract Tetris N1-4
02	mar. 27 oct.	4 h	**Le contrat d'abord**	Contract Tetris N5-8
03	ven. 30 oct.	3 h	**Négocier l'interface**	API Diplomacy N1-3
04	mar. 17 nov.	7 h	**Choisir sa stack**	Stack Duel N1-5
05	jeu. 19 nov.	7 h	**Le jour de l'intégration**	API Diplomacy N4-6 + duels
06	mar. 8 déc.	7 h	**Tenir la ligne**	Tower Defense N1-6
07	mar. 12 janv.	7 h	**L'usine à livrer**	Pipeline Tycoon N1-6
08	lun. 1er févr.	3 h	**Chaque milliseconde**	Stack Duel N6-8
09	mar. 2 févr.	4 h	**Changer sans casser**	Pipeline Tycoon N7-9
10	jeu. 4 févr.	4 h	**Incident en prod**	Tower Defense N7-10
11	ven. 5 févr.	3 h	**Ce qui reste écrit**	API Diplomacy N7-8
12	mar. 23 févr.	4 h	**Le grand tournoi**	Grand tournoi
13	mar. 16 mars	3 h	**Soutenances**	Remise des trophées

La note, c’est la soutenance

Le 16 mars 2027, chaque équipe présente son mini-jeu intégré au hub.

- 30 min de présentation et de **démo en direct depuis le hub**
- 15 min de questions, adressées à n’importe quel membre
- Le dépôt est figé au tag `v1.0.0`

Ce qui est évalué

- L’intégration réelle par le contrat (v2 au 16 mars)
- La coordination visible : PR relues, ADR, contrat versionné
- La qualité : tests, CI, sécurité de la clé de jeu
- La documentation et la traçabilité
- La démo, et vos réponses

L’XP n’est pas la note

Source	XP
Niveau terminé	base × étoiles (base 50 à 150)
Premier passage	+25 %
Échauffement Tetris	10 à 40 par manche, 3 manches par jour
Duel gagné	+60
Jeu d’une équipe	jusqu’à 100 par niveau et par jour

- L’XP mesure **l’entraînement**, pas la compétence finale.
- Rejouer pour améliorer ses étoiles ne rapporte que **la différence**.
- Les scores sont **recalculés côté serveur** : tricher ne rapporte rien.
- Votre mini-jeu, une fois intégré, **donne de l’XP aux autres** : c’est votre premier client.

Front et back : qui porte quoi

Front

- Le rendu, l'accessibilité, la performance **perçue**
- L'état de l'écran : chargement, vide, erreur
- La validation **d'ergonomie** (aider à bien saisir)
- Le cache côté client et sa fraîcheur
- La sécurité du navigateur : XSS, stockage des jetons

Le contrat : formes, codes, erreurs, limites

Back

- Les règles métier et **la source de vérité**
- La validation **qui protège** (le front se contourne)
- Les données, leur cohérence, leurs migrations
- La sécurité : authentification, droits, limites de débit
- Les intégrations et les secrets

La frontière est d’abord organisationnelle

> Un système reproduit la structure de communication de l’organisation qui le conçoit.
>
> Melvin Conway, 1968

- Deux équipes qui se parlent peu produisent une API **bavarde** et **défensive**.
- L’API devient le **canal de communication** officiel : ce qui n’y est pas écrit sera mal compris.
- On peut s’en servir à l’envers : choisir la frontière d’API, puis organiser les équipes autour (manœuvre de Conway inverse).

Trois façons de découper les équipes

Découpage	Pour	Contre
Par couche (front, back)	Expertise profonde, outillage homogène	Chaque fonctionnalité traverse deux files d’attente
Par fonctionnalité (équipes full-stack)	Une équipe livre de bout en bout	Compétences diluées, cohérence d’UI à surveiller
Front + BFF possédé par le front	Le front façonne son API, le back garde le domaine	Un service de plus à opérer

Dans le projet, vous travaillez **par couche, volontairement** :
c'est là que la coordination fait mal, donc là qu'on l'apprend.

Les frictions classiques (1/2)

Friction	Symptôme	Ce qui l'évite
Le champ fantôme	Le front lit `avatarUrl`, le back ne l'envoie jamais	`required` explicite dans le schéma
Le type qui glisse	`"42"` au lieu de `42`, une date sans fuseau	Types et formats (`date-time`, ISO 8601)
Null, absent ou vide	`null`, champ absent, `""` et `[]` traités pareil… ou pas	Nullabilité déclarée, sens de chaque cas
L'erreur improvisée	`200` avec `{ "error": … }`, page HTML sur un 500	Un format d'erreur unique, des codes stables

Les frictions classiques (2/2)

Friction	Symptôme	Ce qui l'évite
La pagination implicite	Le back coupe à 50 éléments sans le dire	Paramètres et métadonnées de page dans le contrat
Le front bloqué	« On attend que l'API existe »	Contrat d'abord, puis mocks
Le changement silencieux	Un champ renommé en production un vendredi	Versionnement, dépréciation, tests de contrat en CI

Chacune de ces frictions a une séance du cours. Vous les vivrez toutes dans votre projet.

Le coût d’un écart dépend du moment où on le trouve

En conception

Une ligne de YAML

Relue dans une PR, corrigée en deux minutes.

À l'intégration

Une demi-journée à deux

Le front et le back cherchent qui a raison, puis réécrivent l'un ou l'autre.

En production

Un incident

Correctif en urgence, données à réparer, post-mortem, confiance entamée.

Un contrat d’API est une promesse vérifiable.

Promesse : ce que chaque côté peut attendre de l’autre. **Vérifiable** : une machine peut dire, sans vous, si elle est tenue.

Ce qu’un contrat fixe, et ce qu’il ne fixe pas

Il fixe

- Les ressources et les opérations (`GET /api/v1/me`)
- La forme exacte des requêtes et des réponses
- Les codes de statut et le format d’erreur
- L’authentification et qui a le droit de quoi
- Les limites : débit, taille, pagination
- La version et le cycle de vie

Il ne fixe pas

- Le langage et le framework de chaque côté
- Le schéma de la base de données
- L’organisation du code
- L’ordre dans lequel le back calcule

Ce qui n'est pas fixé est la liberté de chaque équipe. Plus le contrat est précis à la jointure, plus chacun est libre derrière.

Un vrai contrat : `GET /api/v1/me`

```yaml
paths:
  /api/v1/me:
    get:
      security: [{ playerToken: [] }]
      responses:
        "200":
          content:
            application/json:
              schema: { $ref: "#/components/schemas/Player" }
        "401": { $ref: "#/components/responses/InvalidToken" }
        "429": { $ref: "#/components/responses/RateLimited" }
components:
  schemas:
    Player:
      type: object
      required: [playerId, pseudo, level]
      properties:
        playerId: { type: string, pattern: "^p_[0-9]+$" }
        pseudo: { type: string }
        level: { type: integer, minimum: 1 }
```

Extrait de `contracts/integration-v1.yaml`, le contrat entre **votre** jeu et la plateforme.

- Qui peut appeler : un jeton Bearer
- Ce qui revient si tout va bien
- Ce qui revient sinon, au même format
- La forme exacte, vérifiable par une machine

Tenu ou pas ?

Chaque réponse est-elle conforme au schéma `Player` ?

```
{ "playerId": "p_42", "pseudo": "Neo", "level": 7 }
```

✓ Conforme

```
{ "playerId": 42, "pseudo": "Neo", "level": 7 }
```

✗ playerId doit être une chaîne

```
{ "playerId": "p_42", "pseudo": "Neo" }
```

✗ level est requis

```
{ "playerId": "p_42", "pseudo": "Neo", "level": 7, "avatar": null }
```

✓ Conforme : champ en plus autorisé

Un format par type d’échange

Format	Pour décrire	Outillage typique
OpenAPI 3.1	API HTTP request/response (REST)	Prism, Spectral, Redocly, générateurs de clients
AsyncAPI 3	Messages et événements (WebSocket, files)	Générateurs, validateurs, documentation
GraphQL SDL	Un graphe interrogeable par le client	Apollo, codegen, introspection
Protobuf	Appels gRPC entre services	`protoc`, `buf` (détection de rupture)
JSON Schema	La forme d’un document : la brique commune	Ajv, zod, pydantic

Dans ce cours : **OpenAPI 3.1**, aligné sur JSON Schema 2020-12. On l'ouvre en détail demain.

Soyez stricts à la jointure

> « Sois tolérant dans ce que tu acceptes » cache les bugs au lieu de les corriger.
>
> RFC 9413, *Maintaining Robust Protocols*, 2023

- Un back qui accepte `"level": "3"` en plus de `3` crée un **contrat implicite** que personne n’a écrit.
- Six mois plus tard, un client en dépend et vous ne pouvez plus le retirer.
- Règle du cours : **validation stricte des requêtes** côté back, **erreur explicite** et documentée, jamais de correction silencieuse.

Projet · 10 h 35 – 11 h 20

Atelier · Rétro-contrat

En binôme, **un front et un back**.

01 Regardez l'écran **Classement** du hub (sur votre machine).

02 Déduisez le contrat de l'API qui l'alimente : ressource, paramètres, champs, types, erreurs.

03 Tranchez toutes les ambiguïtés : égalités, joueur sans partie, pagination, tri.

Pas d'outil imposé. Un contrat incomplet mais tranché vaut mieux qu'un contrat complet mais flou.

Livrable de fin de séance

- ☐ Un contrat par binôme, papier ou YAML
- ☐ Chaque champ typé, requis ou facultatif
- ☐ Les erreurs possibles et leur format

Rendu : affiché au mur, photographié

Projet · 11 h 30 – 12 h 20

Atelier · Implémentation à l’aveugle

01 Chaque binôme **donne son contrat** à un autre binôme.

02 Le front écrit l’écran à partir du contrat reçu, sur une réponse mockée.

03 Le back écrit la route qui renvoie ce que le contrat promet.

04 **Interdiction de se parler** : seul le contrat circule.

05 À 12 h 05, on branche le front sur le back. On compte les écarts.

Livrable de fin de séance

- ☐ Un mock front et une route stub back, écrits sans se parler
- ☐ La liste des écarts constatés en les branchant

Rendu : liste des écarts au tableau

Projet · 12 h 20 – 13 h 00

Atelier · Restitution

Chaque binôme présente **ses deux pires écarts** :

- ce qui s’est passé au branchement ;
- la question qu’il aurait fallu poser ;
- **la ligne de contrat** qui l’aurait rendue inutile.

On classe les écarts au tableau selon les sept frictions du matin.

Livrable de fin de séance

- [ ] Pour chaque écart, la ligne de contrat qui l'aurait évité

Rendu : synthèse photographiée et partagée

Le déroulé du jour

Lundi 26 octobre 2026 · 9 h – 13 h · 14 h – 17 h

Matin · 9 h 00 – 13 h 00

9 h 00	Accueil et connexion	10 min
9 h 10	Contract Tetris N1-4	30 min
9 h 40	Théorie 1 · Deux équipes, un contrat	55 min
10 h 35	Atelier · Rétro-contrat	45 min
11 h 20	Pause	10 min
11 h 30	Atelier · Implémentation à l'aveugle	50 min
12 h 20	Atelier · Restitution	40 min

Après-midi · 14 h 00 – 17 h 00

14 h 00 — **Théorie 2 · Architectures et sujet du projet** — 35 min

SPA, REST, WebSocket, BFF : qui porte quoi. Le sujet : un mini-jeu pédagogique intégré au hub par le contrat d'intégration v1.

14 h 35	Constitution des équipes	20 min
14 h 55	Cadrage du mini-jeu	50 min
15 h 45	Pause	10 min
15 h 55	Architecture cible	50 min
16 h 45	Pitch éclair	15 min

Architectures et sujet du projet

Théorie 2 · 14 h 00 · 35 min

01 Où vit le rendu

02 REST

03 WebSocket

04 BFF

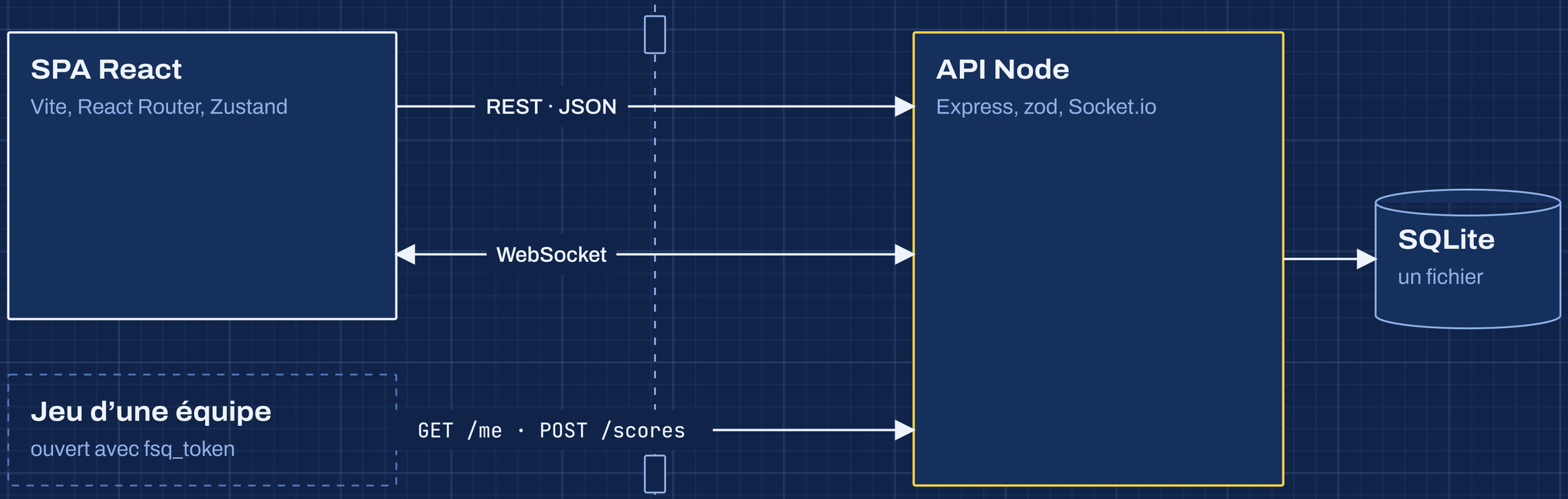
05 Le sujet du projet

Où vit le rendu ?

Modèle	Le HTML est produit…	Le contrat sert à…	Bon choix quand…
SPA	dans le navigateur	tout : chaque écran passe par l'API	application derrière connexion, très interactive
SSR	sur un serveur, à chaque requête	le serveur de rendu, rarement le navigateur	SEO, premier affichage rapide, contenu personnalisé
SSG	une fois, au build	le build, puis plus rien	contenu qui change peu (docs, marketing)
Îlots	page statique + composants hydratés	seulement les îlots interactifs	pages surtout lues, un peu interactives

Le hub Full-Stack Quest est une **SPA** : derrière une connexion, pas de SEO, beaucoup de temps réel.

L’architecture du hub



REST, ce que ça veut dire vraiment

- Des **ressources** nommées par des URL : `/classes/A2/leaderboard`
- Une **interface uniforme** : les verbes HTTP ont un sens
- **Sans état** : chaque requête porte son authentification
- **Cachable** : les réponses disent si on peut les réutiliser

En pratique, l'industrie s'arrête au niveau 2 de Richardson (ressources + verbes). L'hypermédia (niveau 3) reste rare.

Verbe	Sûr	Idempotent
`GET`	✓ oui	✓ oui
`PUT`	✗ non	✓ oui
`DELETE`	✗ non	✓ oui
`POST`	✗ non	✗ non
`PATCH`	✗ non	✗ pas forcément

Un échange REST, en clair

```
POST /api/v1/scores HTTP/1.1
Host: localhost:3000
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9…
X-Game-Key: gk_3f9c…
Content-Type: application/json

{ "gameId": "cache-racer", "level": 3, "score": 1850, "durationMs": 94000 }
```

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "accepted": true, "xpGained": 120 }
```

- Le **verbe** et la **ressource** disent l'intention
- Deux en-têtes, deux identités : le **joueur** et le **jeu**
- `201` : une ressource a été créée
- Corps et réponse : décrits par `ScoreRequest` et `ScoreResponse`

Quand le serveur doit parler en premier

	Polling REST	SSE	WebSocket
Sens	client → serveur	serveur → client	les deux
Transport	HTTP	HTTP, reconnexion native	connexion dédiée, persistante
Coût serveur	requêtes répétées	une connexion par client	une connexion **avec état** par client
Contrat	OpenAPI	format des événements	**à écrire vous-mêmes** (AsyncAPI, types partagés)
Bon pour	données qui changent lentement	flux, notifications, classement en direct	négociation, jeu à plusieurs

Le WebSocket n'a pas de sémantique HTTP : pas de verbes, pas de codes. **Tout est à contractualiser.**

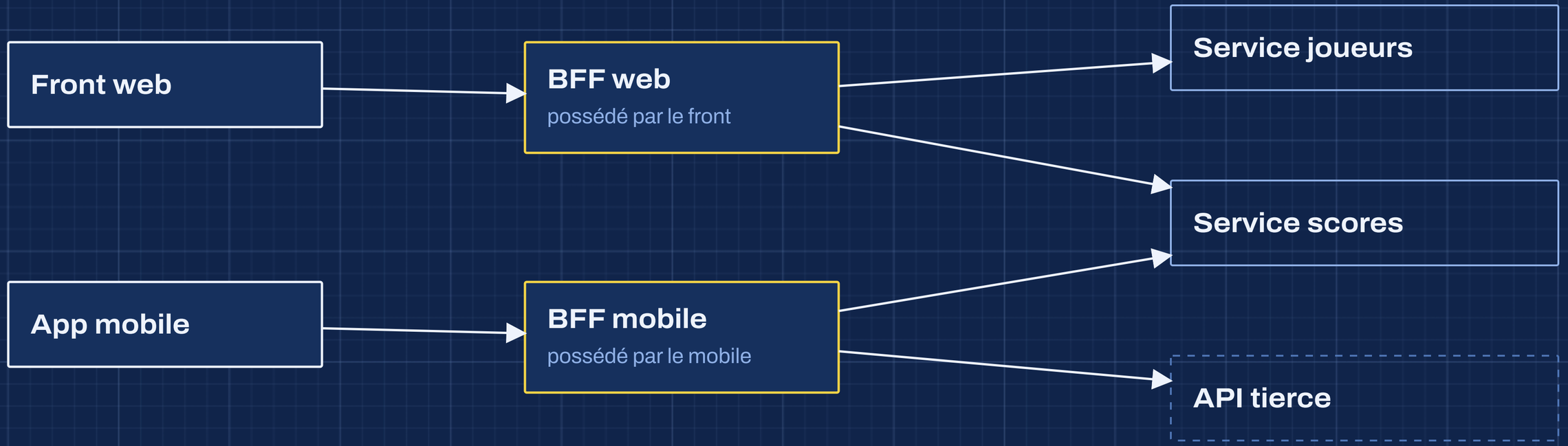
Un contrat d’événements, en TypeScript

```ts
// packages/shared : importé par le front ET le back
export interface ServerToClient {
  "room:state": (s: RoomState) => void;
  "spec:changed": (e: { reason: string; changes: string[] }) => void;
}
export interface ClientToServer {
  "contract:edit": (patch: Patch, ack: (r: Ack) => void) => void;
  "chat:send": (text: string) => void;
}
type RoomState = { version: number; draft: Draft; messagesLeft: number };
type Ack = { ok: true; version: number } | { ok: false; error: string };

// back
const server = new Server<ClientToServer, ServerToClient>(http);
// front
const socket: Socket<ServerToClient, ClientToServer> = io(url);
```

- Les noms d’événements et leurs charges sont **typés des deux côtés**
- Une faute de frappe devient une **erreur de compilation**
- L’accusé de réception (`ack`) rend explicite le succès ou l’échec
- Limite : ne protège que si les deux côtés sont en TypeScript

Le BFF : un back pour un front



Agrège plusieurs appels, **adapte** la forme au besoin de l'écran, **garde les secrets** côté serveur.

Le back de votre mini-jeu est un BFF.

Il garde la `X-Game-Key` hors du navigateur, appelle le hub pour votre front et adapte ses réponses à votre jeu. Si la clé apparaît dans le JavaScript livré, n’importe qui peut envoyer des scores.

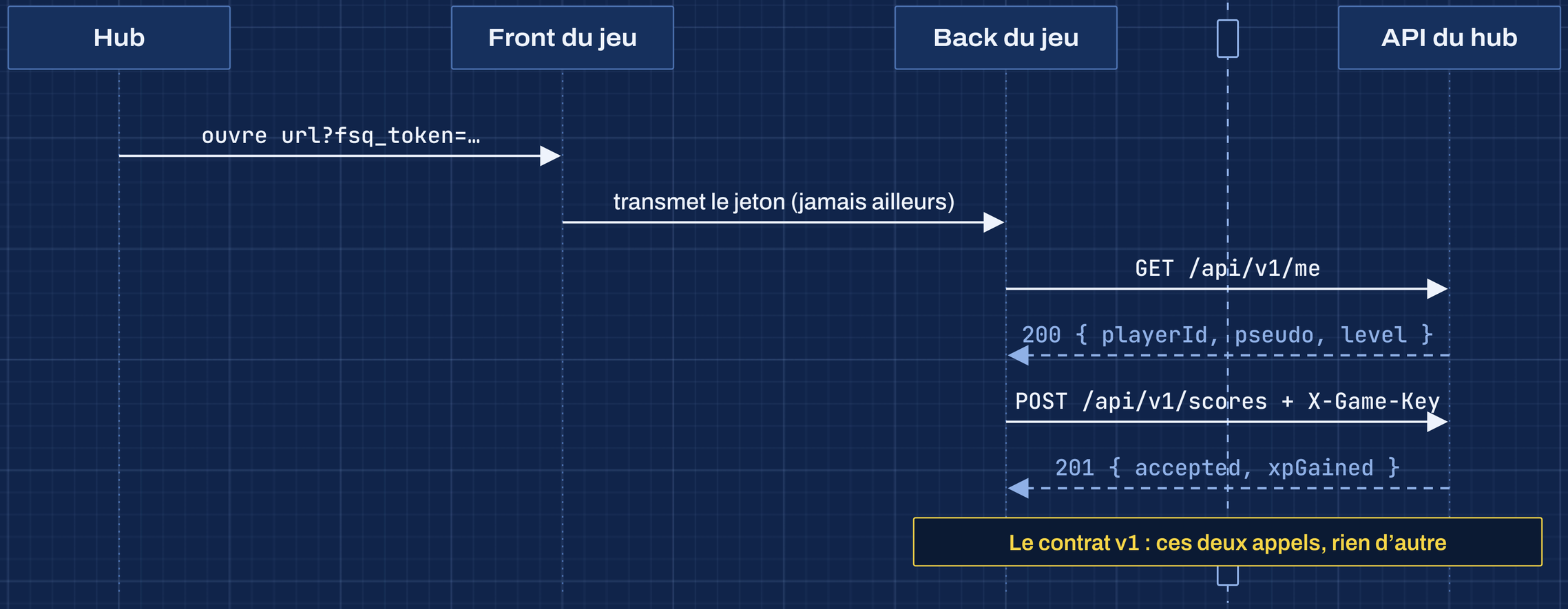
Votre projet : un mini-jeu pédagogique, branché sur le hub par un contrat.

Chaque équipe conçoit un jeu qui fait comprendre **une notion du cours**, le construit en front et back séparés, et l'intègre à Full-Stack Quest **uniquement** à travers le contrat d'intégration v1, comme une équipe qui consomme l'API d'une autre.

Ce qu’on attend du jeu

- **Une notion du cours**, rendue jouable
- **3 à 5 niveaux**, une partie de 5 minutes
- Un **score** et un `scoreMax` déclaré
- Un **front** et un **back** distincts, chacun possédé par son sous-groupe
- Un **contrat OpenAPI** entre vos deux sous-groupes
- L’**intégration au hub** par le contrat v1 (19 nov.)
- La **migration en v2** quand elle sera publiée (2 févr.)
- Un **manifeste** `/fsq-manifest.json` publié
- Une **CI**, des **tests**, un **déploiement**
- Des **ADR** pour chaque choix structurant

Le parcours d’une partie



Les jalons du projet

01	lun. 26 oct.	7 h	**Deux équipes, un contrat**	Lancement, équipes, architecture
02	mar. 27 oct.	4 h	**Le contrat d'abord**	Contrat OpenAPI de l'équipe, mocks
03	ven. 30 oct.	3 h	**Négocier l'interface**	Repo et premier CI
04	mar. 17 nov.	7 h	**Choisir sa stack**	Sprint 1
05	jeu. 19 nov.	7 h	**Le jour de l'intégration**	Intégration au hub, changement de spec
06	mar. 8 déc.	7 h	**Tenir la ligne**	Sécurité, audit croisé
07	mar. 12 janv.	7 h	**L'usine à livrer**	Tests, déploiement en staging
08	lun. 1er févr.	3 h	**Chaque milliseconde**	Test de charge
09	mar. 2 févr.	4 h	**Changer sans casser**	Migration vers le contrat v2
10	jeu. 4 févr.	4 h	**Incident en prod**	Incident simulé, post-mortem
11	ven. 5 févr.	3 h	**Ce qui reste écrit**	Documentation, gel des fonctionnalités
12	mar. 23 févr.	4 h	**Le grand tournoi**	Tests croisés, répétition
13	mar. 16 mars	3 h	**Soutenances**	Soutenances

Les équipes

- **3 équipes** : 4, 4 et 3 personnes
- Dans chaque équipe, un sous-groupe **front** et un sous-groupe **back**
- Un **lead tournant** à chaque séance : il coordonne, il ne décide pas seul
- Le front ne modifie pas le code du back, et inversement : on passe par une **PR relue par l'autre côté**

Le rôle du lead du jour

- Ouvre la séance par 5 min de synchronisation
- Tient à jour le tableau des dépendances
- Remonte les blocages à l'intervenant
- Écrit le compte rendu de 5 lignes en fin de séance

Projet · 14 h 35 – 14 h 55

Constitution des équipes

01 Rejoignez l’équipe annoncée (4, 4, 3).

02 Répartissez front et back. Chacun doit pouvoir **expliquer l’autre côté** en soutenance.

03 Créez le dépôt dans l’organisation de la promo.

04 Désignez le lead du jour.

Livrable de fin de séance

- ☐ Nom d'équipe et répartition front / back
- ☐ Dépôt GitHub créé, intervenant invité
- ☐ Lead du jour désigné

Rendu : nom du dépôt communiqué à l'intervenant

Projet · 14 h 55 – 15 h 45

Cadrage du mini-jeu

- Quelle **notion du cours** le joueur doit-il comprendre en jouant ?
- Quelle **mécanique** la rend tangible : erreur visible, conséquence immédiate ?
- **3 à 5 niveaux** : qu’est-ce qui rend le suivant plus dur ?
- Le **score** : comment se calcule-t-il, quel est son maximum ?

Testez l'idée sur papier avec une autre équipe avant d'écrire une ligne de code.

Livrable de fin de séance

- [ ] `docs/pitch.md` : notion, mécanique, niveaux, score
- [ ] La règle de score et le `scoreMax` déclarés

Rendu : `docs/pitch.md` sur main

Projet · 15 h 55 – 16 h 45

Architecture cible

01 Schéma **front / back / hub** : qui appelle qui, avec quoi.

02 Trajet du `fsq_token` et emplacement de la `X-Game-Key`.

03 Choix techniques **provisoires** : framework front, framework back, stockage.

04 Premier **ADR** : le choix de stack, ses alternatives, ses conséquences.

Livrable de fin de séance

- ☐ Équipes constituées, dépôt GitHub créé, intervenant invité.
- ☐ `docs/pitch.md` : nom du jeu, notion du cours visée, mécanique, niveaux, règle de score.
- ☐ `docs/architecture.md` : schéma front / back / hub et trajet du `fsq_token`.
- ☐ `docs/adr/0001-*.md` : premier ADR (choix de stack provisoire).

Rendu : dépôt Git de l'équipe, branche main

Projet · 16 h 45 – 17 h 00

Pitch éclair

Chaque équipe présente, debout, devant son schéma :

- le nom du jeu et **la notion** qu'il fait comprendre ;
- ce que fait le joueur pendant les 30 premières secondes ;
- **le chemin d'un score**, du clic du joueur à l'XP dans le hub.

Livrable de fin de séance

- ☐ 1 min 30 par équipe, sans slides
- ☐ Le jeu, la notion, le schéma

Rendu : dépôt Git de l'équipe, branche main

Ce qu'on retient

Vu aujourd'hui

- Un contrat d’API est une **promesse vérifiable** entre deux équipes
- Les frictions front/back sont **prévisibles** : champ fantôme, type, nullabilité, erreurs, pagination, blocage, changement silencieux
- Être **strict à la jointure** évite les contrats implicites
- SPA, REST, WebSocket, BFF : **qui porte quoi**, et ce qu’il faut contractualiser
- Votre back de jeu est un **BFF** : la clé reste côté serveur

À rendre

- [ ] Équipes constituées, dépôt GitHub créé, intervenant invité.
- [ ] `docs/pitch.md` : nom du jeu, notion du cours visée, mécanique, niveaux, règle de score.
- [ ] `docs/architecture.md` : schéma front / back / hub et trajet du `fsq_token`.
- [ ] `docs/adr/0001-*.md` : premier ADR (choix de stack provisoire).

Prochaine séance

Mardi 27 octobre 2026

Le contrat d'abord

Contract-first, OpenAPI, mocks

Contract Tetris

Niveaux 5 à 8

Lire un contrat REST d'un coup d'œil